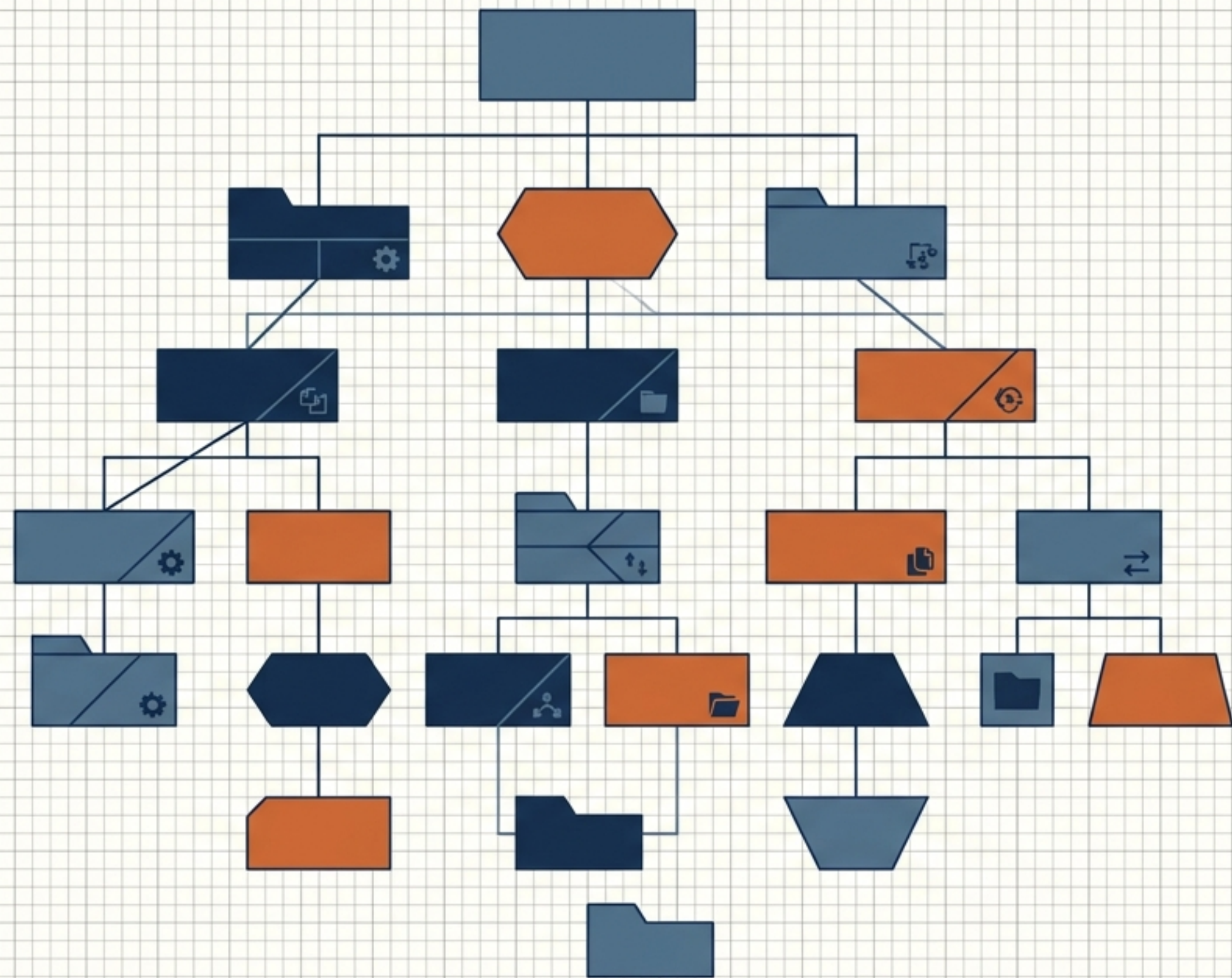




Exam Alert

HTML5プロフェッショナル
認定試験 レベル1 攻略設
計図

マルチメディアとグラフィック
クスAPIの完全マスター

共有・反復学習用
ビジュアルシート



対策教材



Exam Alert
試験対策



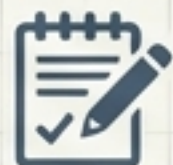
マルチメディア制御 (Multimedia)

- HTMLMediaElement (<audio>, <video>)
- ストリーミング (Streaming)
- MSE (Media Source Extensions)



グラフィックス描画 (Graphics)

- Canvas API (2D/ビットマップ)
- SVG (2D/ベクター)
- WebGL (3D)

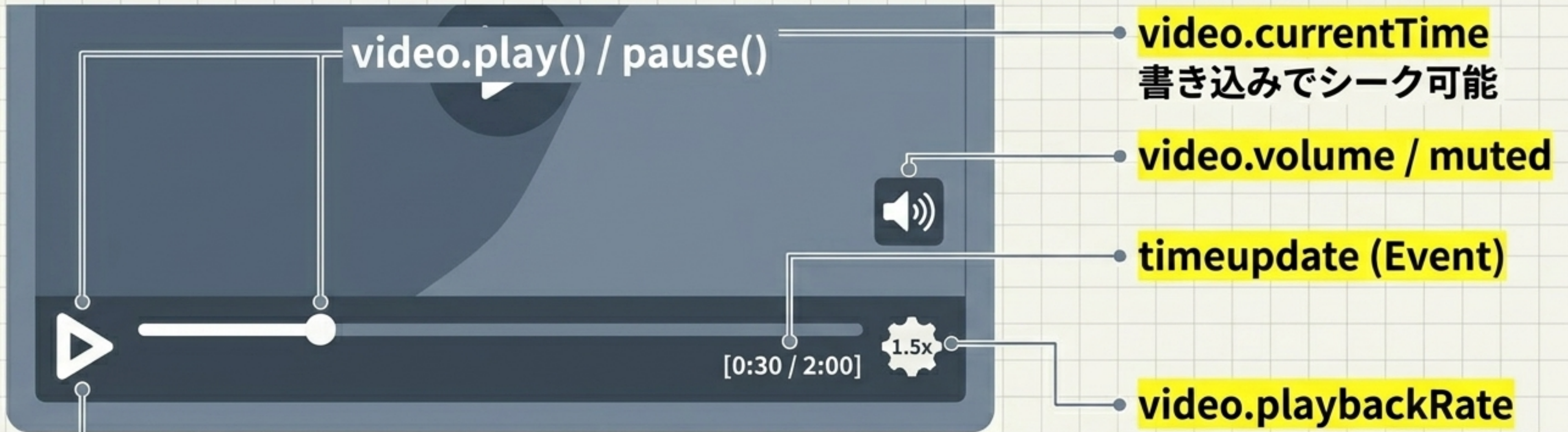


試験対策 (この2領域のAPIの役割と仕様の違いが最重要出題ポイント)

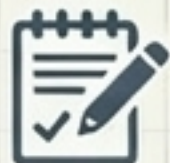
HTMLMediaElementの解剖図



Exam Alert
試験対策



<audio>と<video>要素は共通のインターフェースを持ち、JavaScriptからユーザー操作なしに再生制御が可能。



試験対策 (JSで再生・停止・音量・再生位置を制御できる)

トラップ注意：play() メソッドの非同期処理

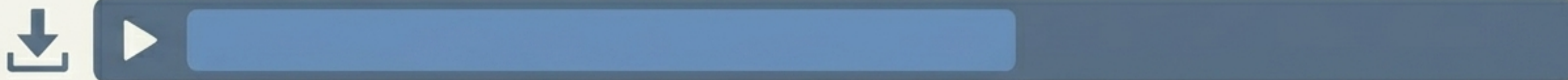


```
video.play().catch(err => console.log('再生できません:', err));
```

 試験対策 (play() は Promise を返す。ブラウザのポリシーによる再生拒否に備えること)

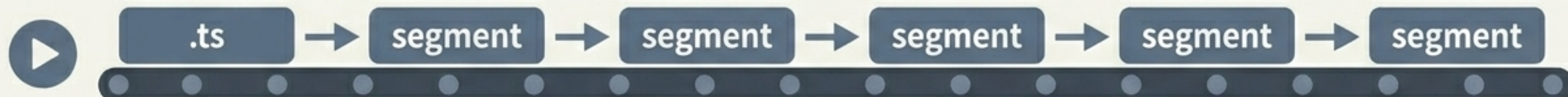
メディア配信のアーキテクチャ：DL vs ストリーミング

プログレッシブダウンロード



先頭から順にダウンロード。ダウンロード済みの範囲のみシーク可能。

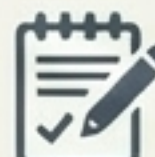
ストリーミング



データを小さなチャンクに分割して順次受信・再生。即時再生や画質自動切替が可能。

HLS / MPEG-DASH:
分割配信の規格。

MSE (Media Source Extensions):
JavaScriptから動画データを動的に
<video> に供給するAPI (ブラウザ実装の基盤)。

 試験対策(ストリーミング=分割順次受信。MSE=JSからの動的供給API)

Canvas API (2D) : JavaScriptで描くピクセルの世界

```
const ctx = canvas.getContext('2d');
```

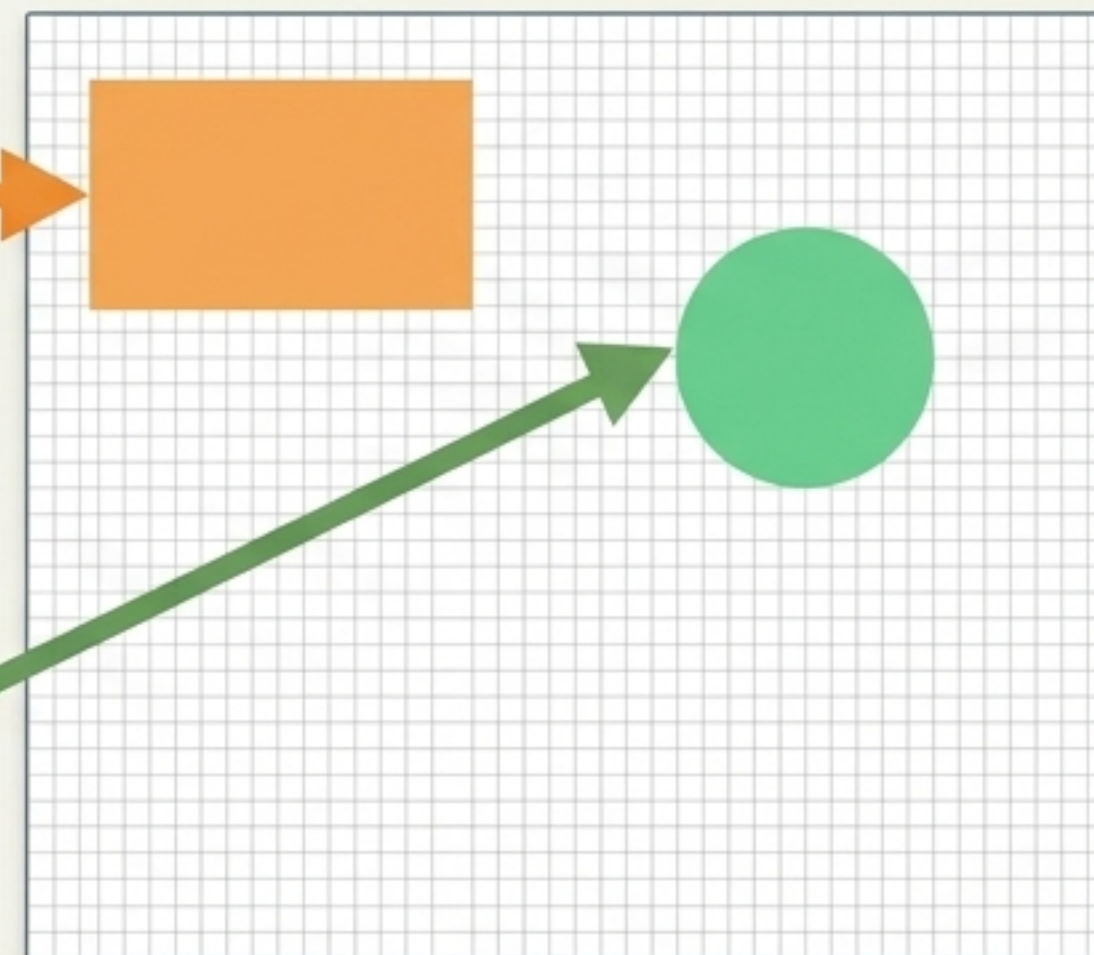
```
ctx.fillStyle = '#f6ad55';
```

```
ctx.fillRect(20, 20, 100, 60);
```

```
ctx.fillStyle = '#68d391';
```

```
ctx.arc(300, 50, 40, 0, Math.PI * 2);
```

```
ctx.fill();
```



⚠ Canvas要素の width / height 属性は、CSSのサイズ指定とは別物。
CSSで変更すると描画が引き伸ばされる。

📝 試験対策(2Dコンテキストを利用し、結果はビットマップ形式として描画される)

SVG : HTMLと一体化するベクターグラフィックス

HTML

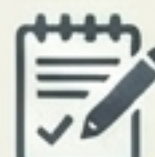
```
<svg width="200" height="100">  
  <circle cx="150" cy="35" r="30" />  
</svg>
```

JS

```
const circle =  
  document.querySelector('circle');  
circle.setAttribute('fill', '#f6ad55');
```

CSS の background-image でも利用可能

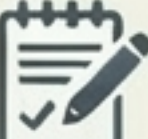
- XMLベースのベクター形式（数式で定義）。
- HTMLに直接 インライン記述可能（<svg>～</svg>）。
- 内部の図形はDOMノードとしてJavaScriptで操作可能。

 試験対策 (インラインSVGの各要素はDOMとして扱われ、操作が可能)

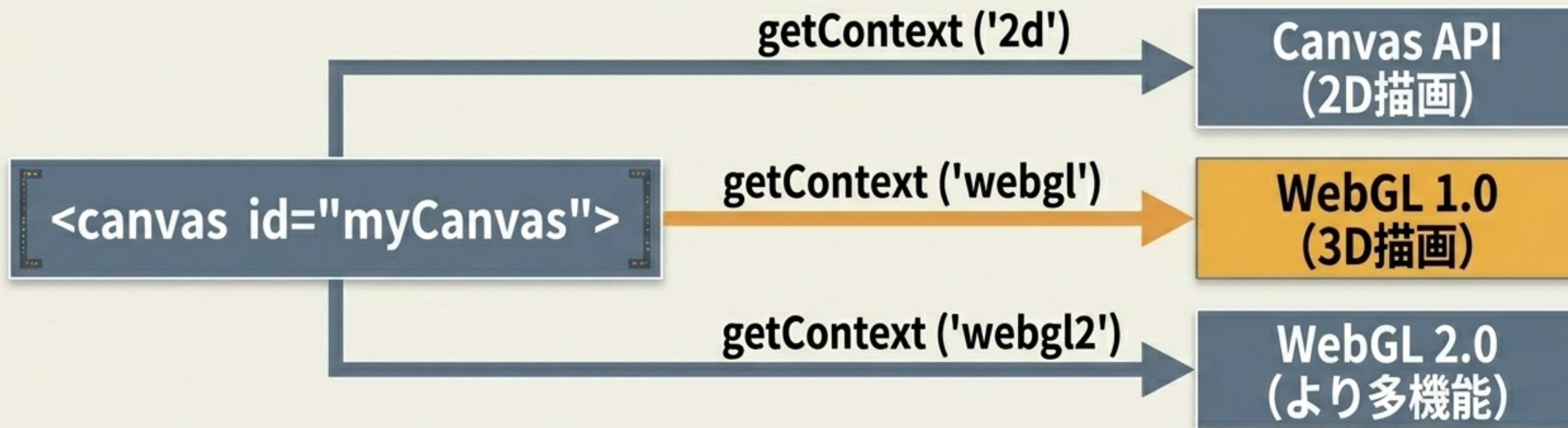
究極の比較：Canvas vs SVG



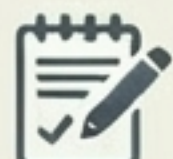
項目	Canvas (2D)	SVG
形式	ビットマップ (ラスター)	ベクター
拡大時	粗くなる (劣化する)	劣化しない (鮮明)
HTML記述	<canvas>要素	インライン記述可能 (<svg>)
DOM操作	不可 (ピクセル単位)	可能 (要素として操作)

 **試験対策** (この4つの違いは最も高い確率で出題される絶対暗記ポイント)

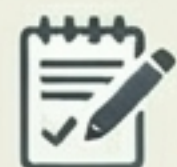
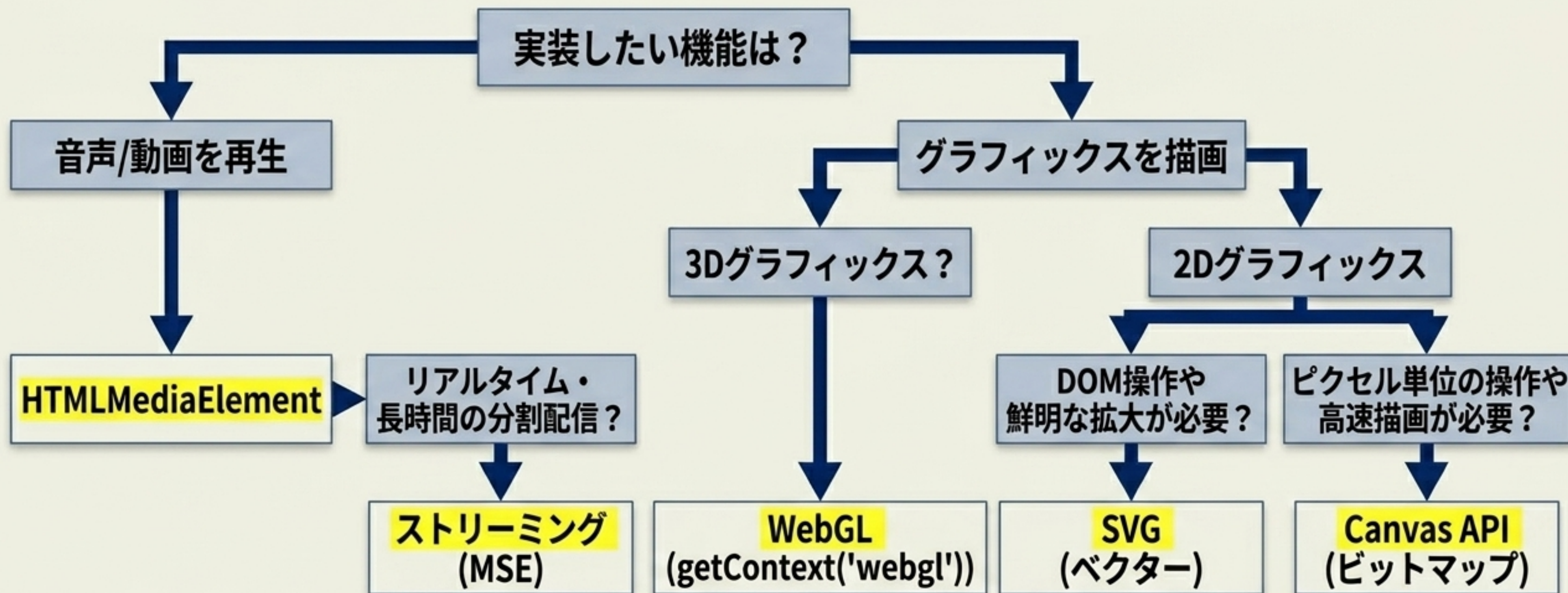
WebGL：ブラウザ上の3Dグラフィックスエンジン



- WebGLは専用のHTML要素を持たない。Canvas要素を使い回す。
- バックエンドは OpenGL ES 2.0 ベース。GPUを活用。

 **試験対策** (WebGLは `<canvas>` 要素に対して `getContext('webgl')` を呼び出す)

API選定のデシジョンツリー (Architecture Map)



試験対策 (要件から適切な技術を逆引きできること)

演習問題：Core Knowledge Check

- Q1: `<canvas>`要素で描画したグラフィックスはベクター形式のため、拡大しても画質が劣化しない。[Y/N]

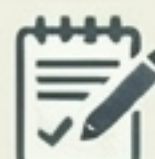
[N] × 誤り。Canvasはビットマップ形式のため拡大すると粗くなる。

- Q2: `<video>`要素の `play()` メソッドの戻り値の型は何か？

Promise型（オートプレイ拒否に備え `.catch()` が必要）

- Q3: Canvas要素の描画サイズを正確に指定するには何を使うか？

`<canvas>`タグの `width / height` 属性（CSSで指定すると引き伸ばされる）

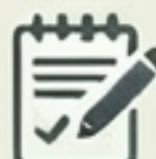
 試験対策 (知識の定着度チェック)

演習問題：Syntax Fill-in-the-Blanks

```
1 // Q1: video要素の音量を50%に設定する
2 video. [ ] = 0.5;
3
4 // Q2: canvasから2Dコンテキストを取得
5 const ctx = canvas.getContext( '[ ]' );
6
7 // Q3: WebGL 1.0のコンテキストを取得
8 const gl = canvas.getContext( '[ ]' );
9
10 <!-- Q4: SVGで円を描く要素 -->
11 <[ ] cx="100" cy="100" r="50" fill="#f6ad55"/>
```

Answer Bank

正解：[volume], ['2d'], ['webgl'], <circle>

 試験対策 (APIの正確なプロパティ名・メソッド名の暗記)

演習問題：Multi-Select Master



<video> に埋め込んだ動画はJSの play() / pause() で制御できる。



HTMLMediaElement の currentTime プロパティに書き込むことでシークできる。



~~Canvasの width・height 属性とCSSのサイズ指定は同じ結果になる。~~

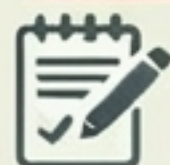
誤り: CSS指定は引き伸ばされる



SVGはHTMLに直接インラインで埋め込め、CSS/JSで操作できる。



~~SVGはビットマップ形式のため、拡大すると粗くなる。~~ 誤り: SVGはベクター形式



試験対策 (正しい記述をすべて選ぶ形式のシミュレーション)

マルチメディア・グラフィックスAPI まとめ一覧

HTMLMediaElement	video要素 と audio要素 をJS制御	play(), currentTime, volume。 JSで再生制御可能
ストリーミング	データを分割順次受信	MSEがブラウザ実装の基盤。 即時再生や画質切替。
Canvas API (2D)	canvas要素 + getContext('2d')	ビットマップ形式（拡大で粗くなる）。 DOM操作不可。
SVG	XMLベースの ベクターグラフィックス	ベクター形式（劣化なし）。 インライン記述可能。DOM操作可能。
WebGL	canvas要素 + getContext('webgl')	OpenGL ES 2.0ベースの 3Dグラフィックス。GPUを活用。

試験本番前の最終確認用チートシート
試験本番前の最終確認用チートシート